WEB 实验报告

姓名：姚嫣如　　**学号**：21080012078　　**班级**：网络空间安全23

实验时间：2026-09-02　　**课程教师**：李晓慧

一、知识点简要总结

（一）二进制基础-从源文件到可执行程序

总体流程：

test.c → 预处理→ test.i → 编译→ test.s → 汇编→ test.o → 链接 → test.exe

1、预处理

- 展开 `#define` 宏定义；
- 处理 `#if`、`#ifdef`、`#elif`、`#else`、`#endif` 等条件编译；
- 处理 `#include`，将被包含文件的内容加入源文件；
- 删除注释；
- 添加行号和文件标识等信息。

输出：`.i` 作为预处理后的输出文件。

2、编译

- 词法分析，拆成一个个 **Token**；
- 语法分析，检查这些 Token 是否符合语言的语法规则，建立类似**语法树**的结构；
- 语义分析，判断含义是否合理；
- 源代码优化；
- 目标代码生成；
- 目标代码优化。

输出：`.s` 作为编译后的输出文件。

3、汇编

把汇编代码翻译成 CPU 真正可以执行的**机器指令**，并打包成可重定位目标程序格式。

输出：`.o` 作为目标文件

4、链接

组织用户程序 + 需要使用的其他目标文件/库，将程序中调用的预编译目标文件等合并进程序。

输出：`.exe` 可执行目标文件。

（二）内存结构

程序被加载运行后，并不是所有内容都混在一整块内存里，而是会按照用途划分为不同区域：

```
高地址
┌──────────────────────┐
|        内核空间       |
├──────────────────────┤
|         栈 Stack     |
|           ↓          |
├──────────────────────┤
|       内存映射区域     |
├──────────────────────┤
|           ↑          |
|         堆 Heap      |
├──────────────────────┤
|        .bss          |
├──────────────────────┤
|        .data         |
├──────────────────────┤
|        .text         |
└──────────────────────┘
低地址
```

1、`.text` 代码段

- 保存程序编译、链接后产生的**可执行代码**；
- 通常具有只读属性，避免程序指令被随意修改；
- 可以被多个程序实例共享，以减少内存占用。

2、`.data` 数据段

分为可读写数据 + 只读数据，可读写数据段：

- 保存**已初始化的全局变量和静态变量**；
- 属于静态内存分配；
- 普通数据区通常可读写。

3、只读数据段

- 保存初始化的字符串常量、常量数据、全局变量等；
- 通常具有只读属性；
- 逆向分析中可以通过程序中的**字符串**辅助判断程序功能和执行逻辑。

4、`.bss` 段

- 保存**未初始化的全局变量和静态变量**；
- 程序运行前由系统初始化为 `0`；
- 属于**静态**内存分配；
- 不需要在可执行文件中完整保存大量初始零值，可以节省文件空间。

5、栈 Stack

- 主要保存函数的**返回地址、函数参数和非 static 局部变量**；
- 运行时动态产生；
- 一般由**高地址向低地址**增长；
- 具有速度快、空间连续、容量相对有限等特点。

6、堆 Heap

- 用于程序运行过程中动态申请内存；
- 内存通常由程序**主动申请和管理**；
- 一般由**低地址向高地址**增长；
- 空间较大，但分配和管理相对栈更复杂。

7、内存映射区域

- 用于文件映射、动态库、匿名映射和共享内存等；
- 可用于加载共享动态库；
- 可以实现进程间共享内存以及高效 I/O。

（三）常见寄存器

1、寄存器

寄存器（Register）是 **CPU 内部用于临时保存数据的高速存储单元**，逆向分析中常通过寄存器判断程序的数据传递和执行流程。

2、x86-64位 中的通用寄存器

- RAX（保存运算结果和函数返回值）→ EAX（32位） → AX（16位）→ AL（8位）
- RBP（栈帧基址寄存器，定位当前函数的**栈帧**）
- RSP（栈指针，指向当前栈顶部）

> 函数调用和局部变量分配会使栈指针发生变化。

- RIP（指令指针寄存器，CPU 当前执行位置附近的指令地址。）

> RIP相对寻址，RIP + 偏移量 → 定位数据

- RDI、RSI、RDX、RCX（常见参数寄存器，**Linux/Unix 下常见的 System V AMD64 调用顺序约定。**）
- RBX、R8、R9、R10、R11、R12、R13、R14、R15（根据需要，变量、地址、中间计算结果）

3、EFLAGS/RFLAGS 标志寄存器

用于记录 CPU **运算和比较**后的状态，条件跳转指令可根据这些标志位决定程序后续执行流程。

- ZF → Zero Flag，零标志
- CF → Carry Flag，进位标志
- SF → Sign Flag，符号标志
- OF → Overflow Flag，溢出标志

（四）链接阶段：静态链接与动态链接

链接用于将不同目标文件及程序使用的库组织起来，解析程序中的**符号**引用并生成最终可执行程序，同时使**分离编译**成为可能。

1、链接

- 将程序自身的目标文件与使用到的其他目标文件、库进行组织和链接；
- 解析函数、全局变量等符号之间的**引用关系**；
- 输出最终**可执行目标文件**；
- 链接可以发生在编译、加载或程序运行过程中。

2、静态链接 Static Linking

在生成可执行程序时，将程序需要的库代码链接到最终可执行文件中。

> 如果两个程序共享一个模块，采用静态链接后，每个可执行文件中都会保存该模块的一份副本；
>
> 同时运行时，磁盘和内存中也可能出现**重复内容**。

- 可能**增加可执行文件大小**，并产生磁盘和内存中的重复资源。

3、动态链接 Dynamic Linking

- 将部分符号解析和重定位过程**推迟**到**程序加载或运行**阶段；
- 程序可以使用**独立存在的共享库**，而不必将所有库代码直接加入自身可执行文件；
- 多个程序可以使用同一个共享模块，减少重复资源；
- 动态链接机制为后续 **GOT、PLT 等地址解析机制**提供基础。

4、动态链接库

- **DLL**（Dynamic Link Library）是 **Windows** 平台常见的**动态链接库**格式；
- **SO**（Shared Object）是 **Linux/Unix** 平台常见的**共享库**格式；
- Windows 可通过 `LoadLibrary`、`GetProcAddress` 等接口动态加载 DLL；
- Linux/Unix 可通过 `dlopen`、`dlsym` 等接口动态加载和访问 SO；
- 共享库可以在程序加载或运行过程中映射到进程地址空间并参与动态链接。

（五）ELF 文件结构

Executable and Linkable Format，可执行与可链接格式，Linux / Unix 类系统中常见的一种**二进制**文件格式。

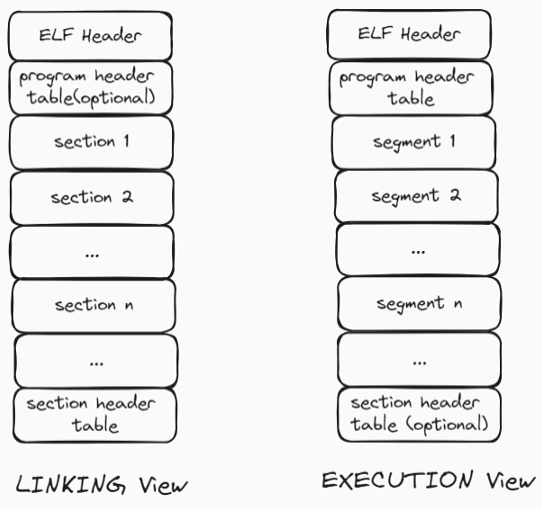
常见 ELF 文件类型：可执行文件、共享库 `.so`、目标文件 `.o`、core 文件。

1、ELF Linking View 链接视图 & **ELF Execution View 执行视图**

链接器关心的内容 → section

程序加载执行时关心的内容 → segment

🍁 **一个 Segment 可以包含一个或多个 Section。**



- Program Header Table ： 程序加载时的说明表。
- Section Header Table：各个 Section 的目录表。

（六）GOT：全局偏移表

GOT 即**全局偏移表**，是 ELF 动态链接机制中用于保存全局变量和外部函数实际引用地址的数据结构。

1、GOT 的作用

- 程序可以通过 GOT 间接获得目标对象的**实际地址**，而不需要在代码中直接写入固定绝对地址；
- 动态链接器可以在程序加载或运行过程中**解析符号地址**，并**更新**相应 GOT 表项；

2、通过 GOT 访问变量

- 程序首先根据 GOT 表项获得目标变量的实际地址，将该地址保存到寄存器后，再通过寄存器**间接**访问变量；
- x86-64 中常结合 `RIP + 偏移量` 的相对寻址方式定位 GOT 表项。

基本过程：

```
程序代码 → GOT 表项 → 变量实际地址 → 访问变量
```

3、`.got` 与 `.got.plt`

均属于 ELF 动态链接相关结构：

- `.got`：主要保存**全局变量**的引用地址；
- `.got.plt`：主要保存**函数**的引用地址，通常与 PLT 配合实现动态函数调用。

4、GOT 的存储属性

GOT 属于数据区域 — **RW 数据段**，即具有**可读、可写**属性；

👉 可写属性使动态链接器能够在运行过程中将解析后的实际地址写入对应表项。

（七）PLT：过程链接表

PLT 即**过程链接表**，主要用于动态链接中的外部函数调用，并与 `.got.plt` 配合实现延迟绑定。

1、PLT 的作用

- PLT 为动态链接程序调用外部函数提供**跳转入口**；
- 外部函数调用通常先进入对应的 `function@plt`，PLT 与 `.got.plt` 配合，根据 GOT 表项中保存的地址跳转到实际函数；
- PLT 负责跳转，GOT.PLT 负责保存地址。

2、外部函数第一次调用

- 第一次调用外部函数时，对应 `.got.plt` 表项中还没有函数的实际地址；
- `function@plt` 将控制权转交给 PLT 的**公共解析入口** `plt0`，`plt0` 进一步调用动态链接器的 `resolver`，动态链接器根据 `.rela.plt` 等重定位信息查找函数实际地址；
- 将解析得到的实际地址写入对应 `.got.plt` 表项，最后跳转到外部函数实际地址执行。

基本过程：

```
function@plt → GOT.PLT → plt0 → resolver → 解析函数地址 → 写回 GOT.PLT → 实际函数
```

👉 第一次需要动态解析，**后续直接使用第一次已经解析并缓存的地址。**

3、延迟绑定 Lazy Binding

外部函数在**第一次被调用时**才进行符号查找和重定位，未被调用的外部函数可以暂时不进行地址解析；

👉 可以减少程序启动阶段不必要的解析工作，加快程序启动速度。

二、实验思路

（一）rev1

程序流程分析 → 定位校验逻辑 → 提取正确输入 → 构造 POST 请求 → 验证结果：

- 使用 IDA 对题目程序进行静态分析，从 `main()` 函数梳理程序整体执行流程，确认程序进入 `phase_1()` 进行登录校验。
- 跟进 `phase_1()`，发现程序分别调用 `check_username()` 和 `check_password()` 对用户名和密码进行验证，分析用户名和密码校验函数，通过交叉引用、字符串或相关变量定位程序中实际用于比较的用户名和密码内容。
- 得到正确的用户名和密码后，根据程序的 POST 参数解析方式构造请求数据，并用Yakit向实验靶机发送 POST 请求获得flag。

（二）rev2

rev1给出提示，该题核心加密思路为**8x8 matrix transpose with rolling XOR**， 即：**8×8 矩阵转置 + 滚动异或**

- 进入 `phase_2`，分析程序 `encrypt` 输出函数，判断转置和异或部分操作流程，并确认运算逻辑。
- 通过交叉引用定位程序中使用的 `keys` 数组，确认该数组长度为 **16 字节**，并在数据段中查看其完整内容。
- 根据程序中的运算逻辑，将提取出的 key 和题目给出的加密数据代入解密程序，复现程序的数据处理过程，从而还原明文得到flag。

（三）rev3

分析主函数 → 确定输入格式 → 还原用户名到密码的变换算法 → Python 复现算法 → 构造合法输入 → 获取 flag：

- 使用 IDA 对程序进行静态分析，从 `main` 函数梳理整体校验流程，确认程序会依次检查 `username`、`password` 的格式以及二者之间的对应关系。
- 分析 `check1() & check()`，确定用户名和密码需要满足的长度限制以及内容限制。
- 根据 `check()` 的运算逻辑编写 Python 脚本，在程序中自行设置一个合法的 `username`，并自动计算对应的 `password`。
- 构造 POST 请求，写入有效的用户名和密码，获取 flag。

（四）ezBase

- 使用 IDA 对 `ezBase.exe` 进行静态分析，首先定位 `main` 函数，确认程序读取用户输入后会调用校验函数判断 flag 是否正确。
- 分析校验逻辑，发现程序会先对输入的 flag 进行编码，再将编码结果与程序数据段中预置的 `cipher` 字符串进行比较，因此推断解题关键是还原编码过程。
- 分析 `encode` 函数中的变换规则，编写脚本对Base64 字符串进行解码，得到原始 flag。

三、实验过程

（一）rev1

1、将文件login 拖入 IDA，并找到`main`函数，`F5`查看伪代码



2、根据实验靶机提示，查看username、password相关信息

> [!] Please login system to get your flag
> [*] Please POST data format: username=&password=

- 函数 `phase_1`：

```
int __fastcall phase_1(char *username, char *password)
{
  char *v2; // rdi

  if ( !(unsigned int)check_username(username) || !(unsigned int)check_password(password) )
    return puts("Login failed :(");
  printf("Login success, congrats and here is the flag: ");
  v2 = getenv("REV_FLAG1_PATH");
  printcontent(v2);
  return phase_2();
}
```

- 函数 `_username`：

```
_BOOL8 __fastcall check_username(const char *username)
{
  size_t v1; // rax

  v1 = strlen(login_name);
  return strncmp(username, login_name, v1) == 0;
}
```

- 函数 `check_username`：

```
IDA View-A
_fastcall check_username(const char *username)
{
  size_t v1; // rax

  v1 = strlen(login_name);
  return strncmp(username, login_name, v1) == 0;
}

; char login_name[]
login_name      db 'superadmin',0
```

👉 要求输入的 `username` 前10个字符等于 `login_name`，即 `superadmin`。

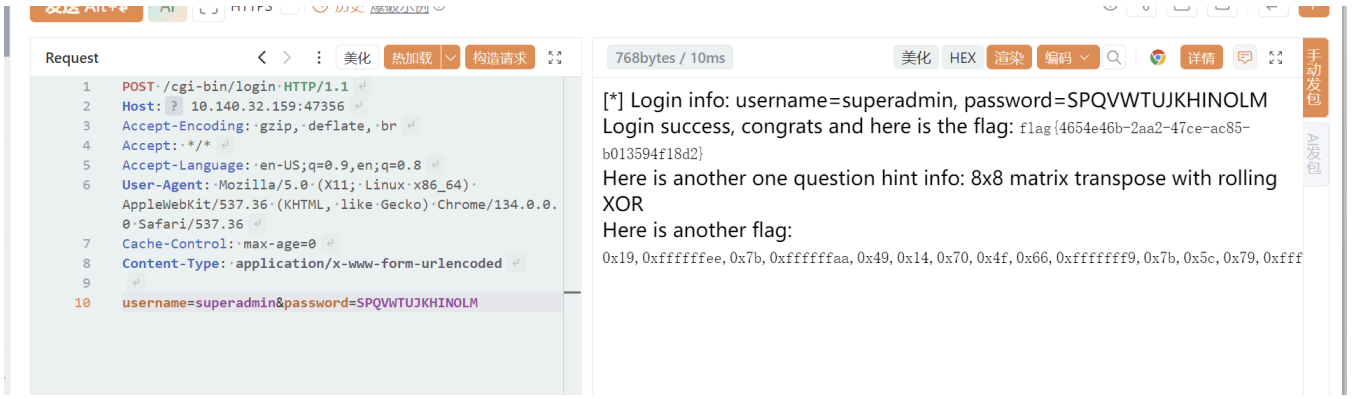
- 函数 `check_password`：

```
__int64 __fastcall check_password(const char *password)
{
  int i; // [rsp+1Ch] [rbp-4h]

  if ( strlen(password) != 15 )
    return 0LL;
  for ( i = 0; i <= 14; ++i )
  {
    if ( password[i] != ((i + 105) ^ 0x3A) )
      return 0LL;
  }
  return 1LL;
}
```

👉 要求 `password` 输入长度为15，值为 （i+105）异或 58 → SPQVWTUJKHINOLM

3、通过 Yakit 提交 POST，获得flag



（二）rev2

1、根据 `phase_1` 函数返回信息，查看 `phase_2`

```
1  unsigned __int64 phase_2()
2  {
3    const char *filename; // [rsp+0h] [rbp-60h]
4    FILE *stream; // [rsp+8h] [rbp-58h]
5    char s[72]; // [rsp+10h] [rbp-50h] BYREF
6    unsigned __int64 v4; // [rsp+58h] [rbp-8h]
7
8    v4 = __readfsqword(0x28u);
9    printf("<br>Here is another one question hint info: 8x8 matrix transpose with rolling XOR");
10   filename = getenv("REV_FLAG2_PATH");
11   stream = fopen(filename, "r");
12   if ( !stream )
13   {
14     puts("[-] Flag file not found!:(");
15     exit(0);
16   }
17   fgets(flag2, 64, stream);
18   fclose(stream);
19   memset(s, 0, 0x40uLL);
20   encrypt(flag2, s);
21   printf("<br>Here is another flag: <br>");
22   dump_hexcode(s, 64LL);
23   return __readfsqword(0x28u) ^ v4;
24 }
```

- 函数 `dump_hexcode` 负责16进制输出缓冲区中变换后的flag2：

```
int __fastcall dump_hexcode(__int64 s, int a2)
{
  int i; // [rsp+1Ch] [rbp-4h]

  printf("<code>");
  for ( i = 0; i < a2; ++i )
    printf("0x%02x,", (unsigned int)*(char *)(i + s));
  return puts("</code>");
}
```

- 函数 `encrypt`：

```
_BYTE *__fastcall encrypt(__int64 flag2, _BYTE *s)
{
  _BYTE *result; // rax
  int i; // [rsp+14h] [rbp-Ch]
  int j; // [rsp+18h] [rbp-8h]
  int k; // [rsp+1Ch] [rbp-4h]

  for ( i = 0; i <= 7; ++i )
  {
    for ( j = 0; j <= 7; ++j )
    {
      result = (_BYTE *)*(unsigned __int8 *)(8 * i + j + flag2);
      s[8 * j + i] = (_BYTE)result;              // 矩阵转置
    }
  }
  for ( k = 0; k <= 63; ++k )
  {
    if ( k )
    {
      result = &s[k];
      *result ^= keys[k % 16] ^ s[k - 1];        // 滚动异或
    }
    else
    {
      result = s;
      *s ^= keys[0];
    }
  }
  return result;
}
```

第一部分进行 8x8 矩阵的转置，核心为`s[8*j+i] = flag2[8*i+j]`。

第二部分进行滚动异或，且keys一共16个字节：

```
C0 = T0 XOR keys[0]
C1 = T1 XOR keys[1] XOR C0
C2 = T2 XOR keys[2] XOR C1
C3 = T3 XOR keys[3] XOR C2
...
```

2、查找`keys`数组信息并记录

```
004020                  public keys
004020 ; _BYTE keys[16]
004020 keys             db 7Fh, 12h, 2Ch, 4Ah, 0Bh, 5Dh, 64h, 3Fh, 45h, 23h, 67h
004020                                          ; DATA XREF: encrypt+9F↑o
004020                                          ; encrypt+F6↑o
00402B                  db 89h, 0ABh, 0CDh, 0EFh, 1
00402B _data            ends
00402B
```

3、编写解密程序，获取flag

```
# 网页给出的 64 字节密文
cipher = [
    0x19,0x3b,0x76,0x05,0x6c,0x55,0x31,0x0e,
    0x27,0x30,0x32,0xffffff88,0x1a,0xffffffaa,0x45,0x44,
    0x5a,0x78,0x79,0x55,0x3b,0x6c,0x08,0x37,
    0x15,0x50,0x03,0xffffffb8,0x23,0xffffffee,0x01,0x00,
    0x04,0x26,0x68,0x0f,0x65,0x38,0x5c,0x63,
    0x15,0x1b,0x44,0xfffffff9,0x36,0xfffffffb,0x14,0x15,
    0x08,0x79,0x64,0x17,0x2c,0x71,0x15,0x2a,
```

```
    0x57,0x16,0x5c,0xffffffb6,0x2c,0xfffffffe1,0x0e,0x0f
]
# 统一转换为真正的单字节
cipher = [x & 0xff for x in cipher]
#  IDA 中的 16 字节 key
keys = [
    0x7f, 0x12, 0x2c, 0x4a,
    0x0b, 0x5d, 0x64, 0x3f,
    0x45, 0x23, 0x67, 0x89,
    0xAB, 0xCD, 0xEF, 0x01
]

# 逆 rolling XOR
# 加密：
# C[0] = T[0] ^ key[0]
# C[k] = T[k] ^ key[k % 16] ^ C[k-1]
# 解密：
# M[0] = C[0] ^ key[0]
# M[k] = C[k] ^ key[k % 16] ^ C[k-1]
M = [0] * 64
M[0] = cipher[0] ^ keys[0]
for k in range(1, 64):
    M[k] = (cipher[k] ^ keys[k % 16] ^ cipher[k - 1])

# 逆矩阵转置
# 转置：flag[8*i+j] = M[8*j+i]
flag = [0] * 64
for i in range(8):
    for j in range(8):
        flag[8 * i + j] = M[8 * j + i]

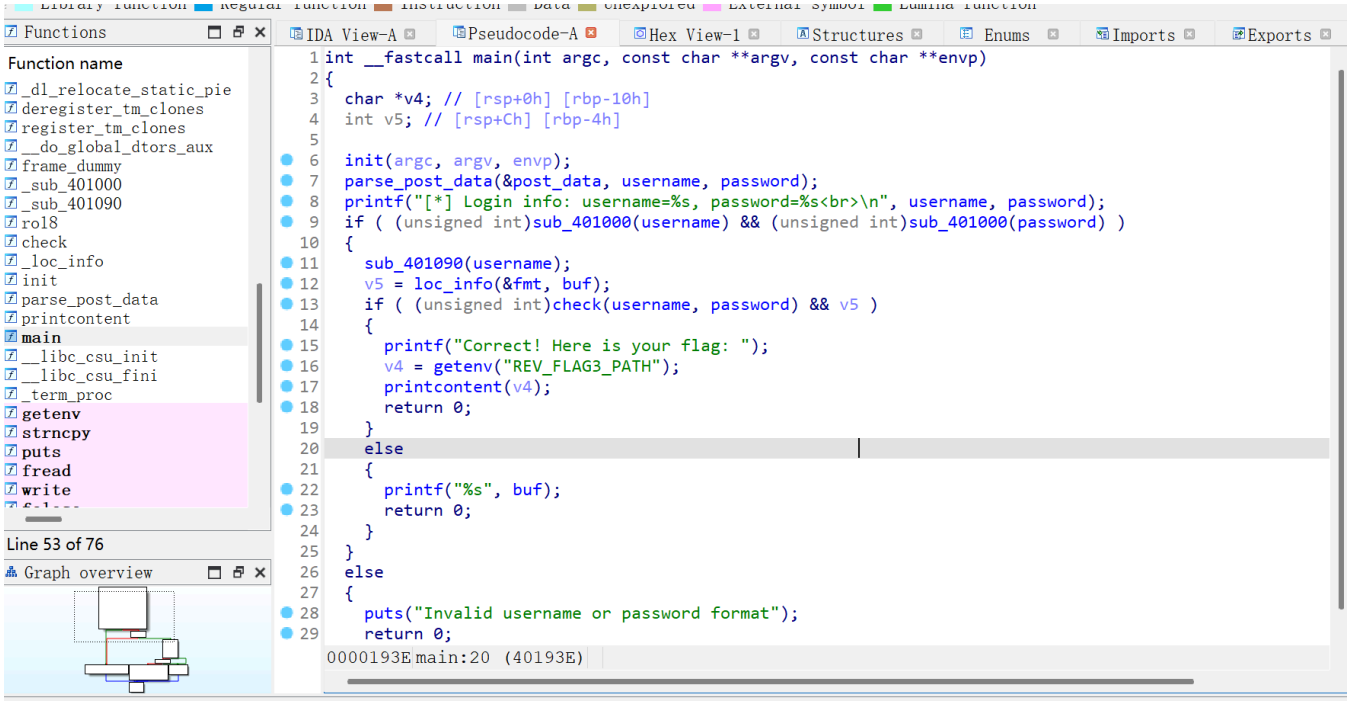
# 去除结尾的 \x00，输出结果
print(bytes(flag).rstrip(b'\x00').decode('utf-8', errors='replace'))
```

4、运行结果

```
(base) rmx08@ISARO:~/study/ns-compractice$ python -u "/home/rmx08/stu
flag{3b8040f0-cbae-4b81-93f2-49cb9e0ad01d}
```

（三）rev3

1、在IDA中打开login2，分析 `main` 函数整体流程，通过对 `username` 和 `password` 进行条件判断，通过之后输出flag。



2、查看判断标准

- 函数 `check1`：格式合法性检验，要求`username/password` → 5 ≤ 长度 ≤ 16，且只包含数字和字母。

```
__int64 __fastcall check1(const char *a1)
{
  size_t v2; // [rsp+10h] [rbp-10h]
  size_t i; // [rsp+18h] [rbp-8h]

  v2 = strlen(a1);
  if ( v2 <= 5 || v2 > 16 )
    return 0LL;
  for ( i = 0LL; i < v2; ++i )
  {
    if ( ((*__ctype_b_loc())[(unsigned __int8)a1[i]] & 8) == 0 )
      return 0LL;
  }
  return 1LL;
}
```

- 函数 `check`：`username/password`须长度一致；对`username`的逐字符异或，并循环左移3位，最后映射成只含有数字和字母的字符串，与password进行比较

```
_BOOL8 __fastcall check(const char *username, const char *password)
{
  char s1[32]; // [rsp+10h] [rbp-40h] BYREF
  unsigned int v4; // [rsp+30h] [rbp-20h]
  unsigned __int8 v5; // [rsp+36h] [rbp-1Ah]
  char v6; // [rsp+37h] [rbp-19h]
  size_t pass_len; // [rsp+38h] [rbp-18h]
  size_t name_len; // [rsp+40h] [rbp-10h]
  unsigned __int64 i; // [rsp+48h] [rbp-8h]

  name_len = strlen(username);
  pass_len = strlen(password);
  if ( pass_len != name_len )
    return 0LL;
  for ( i = 0LL; i < name_len; ++i )
  {
    v6 = username[i];
    v5 = v6 ^ (name_len + 7 * i);
    v5 = rol8(v5, 3LL);
    v4 = v5 % 62u;
    if ( v4 > 9 )
    {
      if ( v4 > 35 )
        s1[i] = v4 + 61;
      else
        s1[i] = v4 + 55;
    }
    else
    {
      s1[i] = v4 + 48;
    }
  }
  s1[name_len] = 0;
  return strcmp(s1, password) == 0;
}
```

3、编写python脚本，构造一个符合条件的`username`，得到`password`。

```python
def rol8(x, n):
    n %= 8
    return ((x << n) | (x >> (8 - n))) & 0xFF
    # 左移n位，拼接上扔掉的前8-n位，最终只保留低8位

def make_password(username):
    name_len = len(username)
    result = []

    for i, ch in enumerate(username):
        # v5 = username[i] ^ (name_len + 7 * i)
        v5 = (ord(ch) ^ (name_len + 7 * i)) & 0xFF
        # ord() 把字符变成整数

        # 循环左移3位
        v5 = rol8(v5, 3)
        # 映射到 0~61
        v4 = v5 % 62
        # 映射到 0-9 / A-Z / a-z
        if v4 <= 9:
            c = chr(v4 + 48)        # 0~9
        elif v4 <= 35:
            c = chr(v4 + 55)        # A~Z
        else:
```

```
            c = chr(v4 + 61)        # a~z
        result.append(c)
    return ''.join(result)      # 拼接成字符串返回

username = "rev3admin"
password = make_password(username)

print("username =", username)
print("password =", password)
```

运行结果：

```
(base) rmx08@ISARO:~/study/ns-compractice$ python -u "/home/rmx08/study/ns-compractice/lab2/rev3.py"
username = rev3admin
password = XlBhY4uUx
```

4、通过 POST 请求，获取flag



（四）ezBase

1、用IDA打开`ezBase.exe`，找到`main`函数，分析代码逻辑流程

- 函数`main`：通过`check_flag`函数进行判断，满足条件则输出 flag。

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char v4[112]; // [rsp+20h] [rbp-70h] BYREF

  _main();
  Init();
  printf("请输入flag: ");
  scanf("%99s", v4);
  if ( check_flag((__int64)v4) )
    puts("验证成功，flag正确！");
  else
    puts("错误的flag！");
  return 0;
}
```

- 函数`check_flag`：利用输入的flag对字符串进行编码，再与cipher进行比较。

  👉 把`cipher`当作已知的目标结果，分析`encode()`的变换规则，然后反推出一个能够使`encode(flag) == cipher`的合法 flag。

```
_BOOL8 __fastcall check_flag(__int64 flag)
{
  char Str1[112]; // [rsp+20h] [rbp-70h] BYREF

  encode(flag, Str1);
  return strcmp(Str1, cipher) == 0;
}
```

```
; char cipher[]
cipher          db 'mZOemISenMjIxseeydLLzMnO0suayM/SnZvPndLKx8eZzp6dysfGzZyC',0
                                        ; DATA XREF: check_flag+23↑o
```

- 函数 `encode`：Base64，每 3 个输入字节编码成 4 个字符。

  每获取3个字节（A、B、C），开始编码，然后使用 `table`将数字转换成字符。

  - a → A高6位，AAAAAA
  - b → A最后2bit+B前4bit，AABBBB
  - c → B低4bit+C高2bit，BBBBCC
  - d → 第三字节的低6位，CCCCCC

  最后处理剩下的长度不满3个字节的特殊情况：补 0 计算，再用 = 补齐4个字符

```
index = 0;
num = 0;
while ( *(_BYTE *)(index + flag) )
{
  v2 = index++;
  v3 = *(_BYTE *)(v2 + flag);
  v4 = num++;
  *(&A + v4) = v3;
  if ( num == 3 )
  {
    a = A >> 2;
    b = ((16 * A) & 0x30) + (B >> 4);
    c = ((4 * B) & 0x3C) + (C >> 6);
    d = C & 0x3F;
    for ( i = 0; i <= 3; ++i )
    {
      v5 = str++;
      *v5 = table[(unsigned __int8)*(&a + i)];
    }
    num = 0;
  }
}
```

```
if ( num > 0 )
{
  for ( j = num; j <= 2; ++j )
    *(&A + j) = 0;
  a = A >> 2;
  b = (16 * A) & 0x30 | (B >> 4);
  if ( num <= 1 )
    v6 = 0;
  else
    v6 = (4 * B) & 0x3C;
  c = v6;
  for ( k = 0; num >= k; ++k )
  {
    v7 = str++;
    *v7 = table[(unsigned __int8)*(&a + k)];
  }
  while ( 1 )
  {
    v9 = num++;
    if ( v9 > 2 )
      break;
    v8 = str++;
    *v8 = 61;
  }
}
*str = 0;
}
```

```
0                   public table
0 table             db '/+9876543210zyxwvutsrqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA',0
0                                       ; DATA XREF: encode+BD↑o
```

table使用逆序的标准Base64字符表。

2、编写py脚本，对cipher进行解码，得到flag

```python
import base64

# 本题自定义 Base64 字符表
custom_table = "/+9876543210zyxwvutsrqponmlkjihgfedcbaZYXWVUTSRQPONMLKJIHGFEDCBA"
# 标准 Base64 字符表
standard_table = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"

# IDA 中找到的 cipher
cipher = "mZOemISenMjIxseeydLLzMnO0suayM/SnZvPndLKx8eZzp6dysfGzZyC"

```

```python
# 将程序使用的自定义 Base64 编码结果还原为原始字符串
def decode_custom_base64(cipher_text):
    # 建立映射：
    # custom_table[i] -> standard_table[i]
    trans_table = str.maketrans(custom_table, standard_table)

    # 把自定义 Base64 转换成标准 Base64
    standard_b64 = cipher_text.translate(trans_table)

    # 标准 Base64 解码
    decoded_bytes = base64.b64decode(standard_b64)

    # 转换为字符串
    decoded_text = decoded_bytes.decode("utf-8")

    return decoded_text

if __name__ == "__main__":
    try:
        flag = decode_custom_base64(cipher)

        print("\n解码得到的 flag:")
        print(flag)

    except Exception as e:
        print("解码失败:", e)
```

```
解码得到的 flag:
flag{ac7798a6-4361-4e70-bd0b-588f1ab5892c}
```

四、实验结果



五、实验心得

通过这次四个reverse实验，我对二进制程序的静态分析流程有了更清晰的认识。实验过程中使用 IDA 从 `main` 函数入手，沿着函数调用和条件判断逐步定位关键校验逻辑，并结合字符串、数组、交叉引用和数据段内容还原程序真实行为。我熟悉了基本的反编译代码阅读和用户名、密码校验分析；重温了学习了如何定位关键数据、提取完整key，并通过编写脚本复现程序中的运算过程；如何分析逐字符异或、循环移位和字符映射等变换，并根据算法正向构造满足条件的输入。

通过这些实验，我体会到逆向分析的关键并不是盲目尝试，而是先理清程序控制流程，再抓住关键函数和数据，最后使用 Python 等工具验证分析结果。这也提高了我阅读反编译代码、还原算法和独立排查问题的能力。